

SRE STUDIUM PRZYPADKU

AZURE ARC HYBRID CLOUD LAB

Czesci 1-4: Kubernetes, Azure Arc, Terraform, CI/CD,
Ingress TLS, HPA, NetworkPolicy, RBAC, GitOps, Trivy, Defender

Autor: Bartosz Suszko

Firma: IT Solutions Bartosz Suszko, Olsztyn

Data: Maj 2026 (Czesci 1-4)

GitHub: github.com/buth11/sre-homelab-azure-arc

Stack: Kubernetes | Azure Arc | Terraform | KQL | Azure DevOps | FluxCD | Trivy | Defender

Spis treści

1. Kontekst i architektura
2. Infrastruktura On-Premise — klaster Kubernetes
3. FinOps — kontrola kosztów
4. Integracja hybrydowa — Azure Arc
5. Observability — Container Insights i Azure Monitor
6. Zapytania KQL — diagnostyka i SLO
7. Load Balancing — weryfikacja round-robin
8. Prometheus i Grafana
9. Infrastructure as Code — Terraform
10. Policy as Code — Azure Policy
11. GitHub — dokumentacja
12. Część 2: Service Principal, Remote State, CI/CD
13. Część 3: Ingress TLS, HPA, NetworkPolicy, RBAC
14. Część 4: GitOps (FluxCD), Trivy, Defender for Cloud
15. Postmortem — POST-001 etcd Split-Brain
16. Wyniki i wnioski
17. Troubleshooting Log (11 incydentów)

Rozdział 1: Kontekst i architektura

Projekt demonstruje budowę produkcyjnego środowiska hybrydowego na GMKtec EVO-X2 (128 GB RAM) łączącego Kubernetes z Microsoft Azure przez Azure Arc. Cztery części praktycznych ćwiczeń z dokumentacją każdego kroku i 11 udokumentowanych incydentów.

Warstwa	Technologia	Zastosowanie
Kubernetes	K3s v1.35.4+k3s1	Klaster 3 węzły
Ingress	Traefik + TLS	HTTPS routing po domenie
Autoscaling	HPA (autoscaling/v2)	Skalowanie CPU 2-8 replik
Network Security	NetworkPolicy	Firewall wewnątrz klastra
Access Control	RBAC + ServiceAccount	Least privilege
GitOps	FluxCD 2.8.8	Synchronizacja z Git
Security Scanning	Trivy 0.70.0	Skanowanie CVE obrazów
Runtime Security	Defender for Cloud	Wykrywanie zagrożeń Microsoft
Cloud Bridge	Azure Arc	Hybrydowa platforma kontrolna
Monitoring	Azure Monitor + Prometheus	Observability
IaC	Terraform + Remote State	Infrastructure as Code
CI/CD	Azure DevOps	Pipeline z approval gate

Rozdział 2: Infrastruktura On-Premise

Wezel	IP	CPU	RAM
k3s-master (Control Plane)	192.168.122.10	4 vCPU	8 GB
k3s-worker1	192.168.122.11	4 vCPU	16 GB
k3s-worker2	192.168.122.12	4 vCPU	16 GB

Rozdział 3: FinOps

Budżet Azure Cost Management skonfigurowany PRZED uruchomieniem jakichkolwiek zasobów — zasada 'FinOps first'. Budżet: 5 USD/mies. z alertami email przy 50% i 100%. Zdefiniowany w Terraform — wersjonowany jak każdy inny kod. Wynik po całym projekcie: 0.00 USD — wszystko mieści się w darmowych limitach Azure.

Rozdział 4: Azure Arc

Agent Azure Arc zainstalowany na klastrze K3s jedna komenda (az connectedk8s connect). Klaster pojawil sie w portalu Azure jako zasob — K3s-Homelab Connected, v1.35.4+k3s1, 3 wezly, 12 rdzeni. Agent nawiazuje polaczenie wychodzace HTTPS/443 z mTLS — nie trzeba otwierac portow przychodzacych. Po restarcie VM klaster reconnectowal sie automatycznie w ciagu 3 minut.

Rozdział 5: Observability

Container Insights (AMA — Azure Monitor Agent) zainstalowany jako rozszerzenie Arc — 4 Pody DaemonSet na klastrze. Weryfikacja: Pody AMA Running, 0 restartow. Dashboard Azure Monitor: CPU avg 1.58%, RAM avg 6.27%, 3 wezly Ready, 29 podow Running.

Rozdział 6: KQL

5 produkcyjnych zapytan KQL napisanych i zweryfikowanych: - KQL-1: Inwentarz wezlow — 3 Ready, v1.35.4+k3s1 - KQL-2: Status Podow per namespace — azure-arc 12 Running, default 3, kube-system 14+2 - KQL-3: Eventy 24h — 13 zdarzen: Rebooted, Unhealthy (chwilowe), FailedScheduling - KQL-4: Pody z restartami — 27 po restarcie VM, wszystkie Running - KQL-5: SLO — 100% dostepnosc (184/184 probek Ready)

Rozdział 7: Load Balancing

Weryfikacja round-robin: 20 requestow na 3 pody — wszystkie 3 Pody obsluzily ruch. 900 rownolegla bez bledow. K3s Klipper Load Balancer rozklada ruch rowno po wszystkich wezlach.

Rozdział 8: Prometheus & Grafana

kube-prometheus-stack zainstalowany przez Helm w namespace monitoring. Stack: Prometheus, Grafana, Alertmanager, kube-state-metrics, node-exporter. Dashboard Grafana: CPU 2.20%, RAM 12.9%, CPU Quota per namespace. Reguly alertow: High CPU (>80%), High Memory (>85%), Pod down >5 minut. Test alertu: zabicie Poda triggerowalo alert w ciagu 2 minut przez Alertmanager.

Rozdział 9: Terraform IaC

5 zasobow Azure zdefiniowanych w kodzie HCL: Resource Group, Log Analytics Workspace, budżet kosztow, Action Group (alerty email), regula alertu OOMKill. Remote state w Azure Blob Storage (sretfstate5496) z blokowaniem i wersjonowaniem. Terraform import dla brownfield deployment. Terraform Plan: 6 zasobow, 0 zmian po stabilizacji. Terraform Apply: 3 dodane, 1 zmieniony.

Rozdział 10: Azure Policy

2 polityki przypisane przez Azure CLI (nie portal — powszechna metoda w produkcji): - Polityka 1: No root containers — żadna aplikacja nie może działać jako root - Polityka 2: Require CPU and memory limits — CPU max 500m, RAM max 512Mi Tryb: DoNotEnforce (Audit) — standardowe podejście na istniejących środowiskach. Wynik: obydwie polityki Compliant — 100% resource compliance.

Rozdział 11: GitHub

Publiczne repozytorium: github.com/buth11/sre-homelab-azure-arc Jezyki: Shell 58.4%, HCL 41.6%.
Autentykacja kluczem SSH — bez tokenow. Kazda zmiana infrastruktury zapisana jako commit Git z autorem i data.

Rozdział 12: Część 2 — Service Principal, Remote State, CI/CD

Remote State — plik sre-homelab.tfstate w Azure Blob Storage:

Plik: sre-homelab.tfstate — 14.53 KiB, szyfrowanie server-side, wersjonowanie aktywne. State lock widoczny w logach: 'Acquiring state lock' / 'Releasing state lock' przy każdym terraform plan.

Pipeline Azure DevOps CI/CD:

Variable Group terraform-credentials: ARM_CLIENT_ID, ARM_CLIENT_SECRET (ukryty), ARM_SUBSCRIPTION_ID, ARM_TENANT_ID. Environment production: Approval Gate aktywna — wszyscy approverzy muszą zatwierdzić (timeout 30 dni).

Pipeline kompletny — oba stage zielone, łącznie 4m 24s. Stage 1 (37s): validate + plan + publikacja artefaktu. Stage 2 (49s): terraform apply po zatwierdzeniu przez Approval Gate.

Rozdział 13: Część 3 — Zaawansowane operacje Kubernetes

Ingress z TLS — routing HTTPS po domenie

Traefik Ingress Controller skonfigurowany do obsługi ruchu HTTPS. Self-signed certyfikat TLS z SAN wygenerowany przez openssl, zapisany jako Secret Kubernetes (kubernetes.io/tls). Routing: demo.sre-lab.local → aras-demo-app:8080.

Weryfikacja w przeglądarce Firefox: X-Forwarded-Proto: https, X-Forwarded-Server: traefik-9bcd9bd9-z2mnj, X-Forwarded-Port: 443.

HPA — automatyczne skalowanie na podstawie CPU

HPA skonfigurowany: minReplicas: 2, maxReplicas: 8, targetCPU: 50%. Test obciążeniowy: ab -n 50000 -c 50 — 1621 req/s przez 30 sekund.

Scale-up: CPU 860%/50% → 8 replik (max). Scale-down: CPU 6%/50% po 5 minutach → 2 repliki (min). Cooldown period działa zgodnie z oczekiwaniami.

NetworkPolicy — firewall wewnątrz klastra

Polityka deny-all dla aras-demo-app. Zezwolone: kube-system (Traefik), monitoring (Prometheus), repliki aras-demo-app. Zablockowane: wszystko inne.

Weryfikacja: nieautoryzowany Pod → timeout/blokada ('ZABLOKOWANY — NetworkPolicy działa'). Traefik → odpowiedź aplikacji.

RBAC — least privilege dla aplikacji

ServiceAccount: aras-demo-sa. Role: get/list/watch na Podach i ConfigMapach — nic więcej. RoleBinding łączy SA z Role.

kubectl auth can-i list pods --as=system:serviceaccount:default:aras-demo-sa → yes
kubectl auth can-i delete pods --as=system:serviceaccount:default:aras-demo-sa → no
Zasada least privilege potwierdzona.

Rozdział 14: Część 4 — GitOps, Trivy, Defender for Cloud

FluxCD — synchronizacja klastra z Git

FluxCD 2.8.8 bootstrapowany z repozytorium GitHub przez klucz SSH. Cztery kontrolery zainstalowane w namespace flux-system: helm-controller, kustomize-controller, notification-controller, source-controller — wszystkie zdrowe.

Aplikacja demo wdrożona do namespace gitops-demo. Flux Kustomization obserwuje katalog k8s/part4/apps/ i synchronizuje co 1 minute.

Test GitOps: zmiana replik z 2 na 3 w deployment.yaml, commit i push do GitHub. Flux automatycznie zastosował zmianę — trzeci Pod pojawił się bez żadnego kubectl apply. Git jest jedynym źródłem prawdy.

Trivy — skanowanie obrazów kontenerowych pod kątem CVE

Trivy 0.70.0 zainstalowany. Skanowany obraz: traefik/whoami:latest (stdlib Go v1.24.1).

Severity	Liczba	Kluczowe CVE
CRITICAL	1	CVE-2025-68121 — crypto/tls: nieprawidłowa walidacja certyfikatu
HIGH	12	CVE-2025-22874, CVE-2025-61726, CVE-2026-25679 i inne
MEDIUM	25	Różne problemy net/http, os, crypto/x509
LOW	2	Problemy modułu net/http, os

Pełne wyniki skanowania zapisane w k8s/part4/trivy/ (format JSON i tabelaryczny). Przyczyna: przestarzałe stdlib Go v1.24.1 — poprawione w v1.24.13+.

Defender for Cloud — bezpieczeństwo runtime

Defender for Containers włączony na poziomie Standard (30-dniowy trial). Rozszerzenie microsoft.azuredefender.kubernetes zainstalowane na klastrze Arc — provisioningState: Succeeded.

7 Podów Defendera działa w namespace mdc: - microsoft-defender-collectors-ds-* (3x — jeden na każdym węzle) - microsoft-defender-pod-collector-misc (1x) - microsoft-defender-publisher-ds-* (3x — jeden na każdym węzle) Wszystkie Running, 0 restartów.

Rozdział 15: Postmortem POST-001

Pole	Wartosc
Severity	P2
Czas	Ponizej 7 minut
Root Cause	Literowka: ks3-worker1 zamiast k3s-worker1
Status	Rozwiazany — regex validation w skryptach

Rozdział 16: Wyniki i wnioski

Metryka	Czesc 1	Czesc 2	Czesc 3	Czesc 4
Wezly	3/3 Ready	—	—	—
SLO	100%	—	—	—
Koszty	\$0.00	\$0.00	\$0.00	\$0.00
Terraform state	Local	Remote (Blob)	—	—
CI/CD	—	Azure DevOps 4m24s—	—	—
Ingress TLS	—	—	HTTPS verified	—
HPA	—	—	3->8->2 replik	—
NetworkPolicy	—	—	Zablokowane	—
RBAC	—	—	Least privilege	—
GitOps	—	—	—	FluxCD wdrozony
Skanowanie	—	—	—	1 CRITICAL, 12 HIGH
Bezpieczenstwo	—	—	—	Defender (3 wezly)
Incydenty	6	8	10	11

Rozdział 17: Troubleshooting Log

ISSUE-001 — KQL: CPUCapacityNanoCores column missing

Symptom: project operator: Failed to resolve scalar expression named CPUCapacityNanoCores

Rozwiązanie: Użyto KubeNodeInventory | getschema do weryfikacji dostępnych kolumn. Uproszczono zapytanie.

ISSUE-002 — Azure Policy: MissingPolicyParameter cpuLimit/memoryLimit

Symptom: (MissingPolicyParameter) policy assignment k8s-resource-limits is missing cpuLimit, memoryLimit

Rozwiązanie: Dodano --params z cpuLimit: 500m i memoryLimit: 512Mi do az policy assignment create.

ISSUE-003 — Terraform: Resource Group already exists (brownfield)

Symptom: Error: A resource with the ID ../SRE-Lab-RG already exists

Rozwiązanie: terraform import azurerm_resource_group.sre_lab /subscriptions/.../SRE-Lab-RG

ISSUE-004 — Terraform: Saved plan is stale after import

Symptom: Error: Saved plan is stale — state was changed after plan was created

Rozwiązanie: Nowy terraform plan -out=tfplan po każdej operacji import.

ISSUE-005 — Container Insights missing after VM restart

Symptom: Pody AMA znikły z kube-system namespace po pełnym restarcie VM

Rozwiązanie: az k8s-extension create --name azuremonitor-containers — agenty wróciły w 37s.

ISSUE-006 — POST-001: etcd Split-Brain (ks3 vs k3s)

Symptom: FATA[0034] Node password rejected, duplicate hostname

Rozwiązanie: kubectl delete node + rm /etc/rancher/node/ + restart k3s-agent. Skrypty zaktualizowane o regex validation.

ISSUE-007 — Azure DevOps: TerraformInstaller task missing

Symptom: A task is missing: TerraformInstaller

Rozwiązanie: Instalacja z Marketplace: ms-devlabs.custom-terraform-tasks.

ISSUE-008 — Pipeline: permission for Variable Group

Symptom: This pipeline needs permission to access a resource

Rozwiązanie: View → Permit — jednorazowe zatwierdzenie na pipeline.

ISSUE-009 — sudo does not expand ~ in paths

Symptom: error: Cannot read file ~/ingress/server.crt: no such file or directory

Rozwiązanie: Przyczyna: sudo uruchamia jako root z innym katalogiem domowym (/root/). Używać pełnej ścieżki /home/buth11/.

ISSUE-010 — HPA shows cpu: unknown — missing resource requests

Symptom: NAME TARGETS MINPODS MAXPODS REPLICAS — cpu: /50%

Rozwiązanie: Dodano resources.requests do Deployment: cpu: 10m, memory: 32Mi. Po rolling update HPA pokazuje poprawne wartosci.

ISSUE-011 — RBAC RoleBinding: apiRef zamiast apiGroup

Symptom: Kubernetes odrzucił obiekt RoleBinding jako nieprawidłowy — pole apiRef nie istnieje

Rozwiązanie: Prawidłowe pole to apiGroup: rbac.authorization.k8s.io w sekcji roleRef. Poprawiono w osobnym pliku YAML.